More on Input/Output

CSE 101 Section 03

Introduction to Algorithms and Computing

Fall 2022

© Royal Military College of Canada

What we have seen so far about input/output in Matlab

- The `input()` function is used to get input in MATLAB

```
% entering an integer
input("Enter an integer : ")
% entering a string
input("Enter a string : ", "s")
```

Used to specify that we expect a string

What we have seen so far about input/output in Matlab

- The `disp()` function is used to output data in MATLAB

```
% output a string
disp("An informative text")

% output a variable
var = 10;
disp(var)
```

Why do we need more?

- The two functions `input()` and `disp()` are sufficient in many scenarios.

- But there are situations where we may need to do things a bit differently.

- Examples:
  - For input, we may need to get the data from a file rather than from the user (discussed later in the course).
  - For output, we may want to have more control over how the results are displayed (more on the next slides).

Input with `disp()`

- The **disp()** function is very easy to use but provides limited control of output formatting.
- The **disp()** function displays exactly one value/variable to the screen.
- The **disp()** function is particularly convenient for displaying vectors, matrices, and complex numbers because it displays the entire array with a single function call.
- Some simple examples:

```
disp(5)
disp(pi)
disp(3.1415926)
myArray = [1 2; 3 4];
disp(myArray)
disp('hi there')
```

Input with `disp()`

- You can create a single string from multiple strings using **string concatenation**.
- The MATLAB function **num2str()** will take a number and convert it into a string. Notice that the brackets **[ ]** create the single string, and are required.

- For example:

```
disp( ['pi = ' num2str(pi)] )
Distance = 120;
disp( ['It is ' num2str(Distance) ' miles to the river.'] )
```

- In general, use **disp()** when you do not care about the exact format of the output

Input with `fprintf()`

- The **fprintf()** function is more complicated to use but it allows you precise control over the output formatting.

- The **fprintf()** function can receive one or more arguments (unlike the **disp()** function).

- Simple examples:

```
fprintf('Hello');

myvar = 123
fprintf('hello = %d', myvar);
```

format specifier

Input with `fprintf()` – *format specifiers*

- A *format specifier* begins with the % character and is followed by other characters that precisely describe how the corresponding output value will be formatted.
- A *format specifier* always contains a single letter (*format Descriptor*) that describes the data type of the output value. The most commonly used data types are:

`%d`	display the value as an integer (no fractional part)
`%f`	display the value in floating point format (3.451)
`%s`	display the value as a string - a series of characters
`%c`	display the value as a single character

Input with `fprintf()` – *escape sequences*

- Special *escape sequences*, beginning with the backslash '\' character, are used to represent "non-printable" characters.

- The most commonly used escape sequences are:

`\n`	a "carriage return" -- it starts a new line of output (or ends the current line of output)
`\t`	a "tab" -- spaces over to the next tab stop

- Examples:

```
Age = 20;
fprintf('I am %d years old.\n', Age)
```

format specifier

escape sequence

Input with `fprintf()` – *examples*

```
fprintf('This is a string with no data values.')

fprintf('This is a string followed by a new line character.\n')

Age = 20;
fprintf('I am %d years old.\n', Age)

fprintf('pi to 2 decimal places = %0.2f\n', pi)

fprintf('pi to 10 decimal places = %0.10f\n', pi)

Name = 'Fred';
fprintf('My name is %s and I am %d years old.\n', Name, Age)
```

Exercise

- Write a program that asks the user for their age and responds by telling them how old they will be in 5 years.

- Here is an example of what the program should *exactly* display.

```
How old are you? 19
In 5 years, you will be 24!
   Bye now!
```